#ccc_r11

私がTDD出来ないのはどう考えてもお前らが悪い！

～エンタープライズJava開発におけるTDD適用の勘所～

2014/11/15

大中浩行

アジェンダ

- なぜTDDするのか
- どうTDDするか
- 技術的な各論
- なぜTDDするのか(again)

Copyright 2014 Hiroyuki Ohnaka

- 大中浩行(Ohnaka,Hiroyuki)
- TDDBC横浜(2011～2013)主宰
- yokohama.devtesting / devtesting-ja
- #agilesamurai #横浜道場
- Twitter @setoazusa

Copyright 2014 Hiroyuki Ohnaka

- グロースエクスパートナーズ(株)
  - 大手通信事業者の法人向けサービス開発プロジェクト
  - TDDの導入、DevOpsの推進、プロジェクトメンバーのフォローなどがお仕事。

Copyright 2014 Hiroyuki Ohnaka

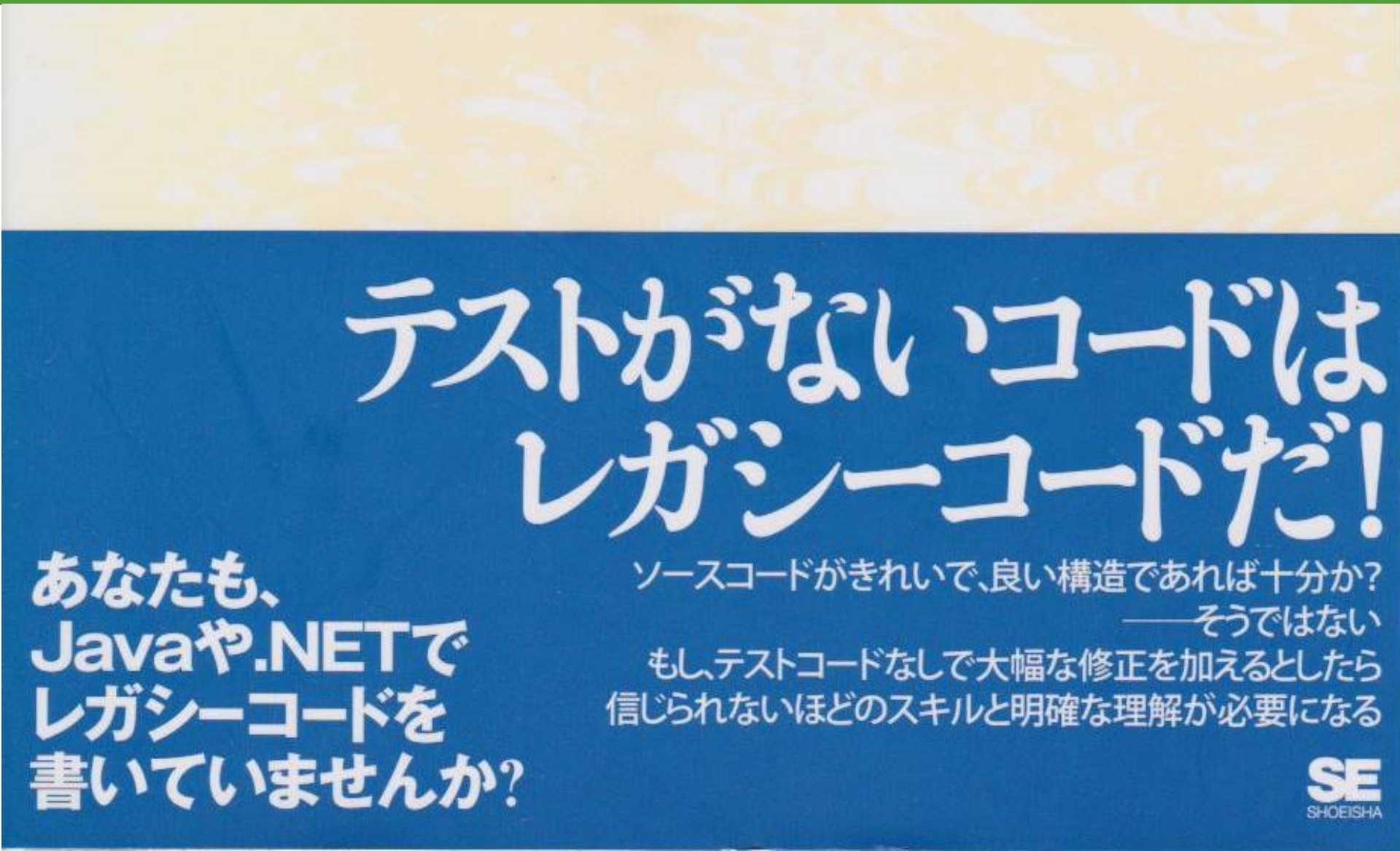
アジェンダ

- **なぜTDDするのか**
- どうTDDするか
- 技術的な各論
- なぜTDDするのか(again)

Copyright 2014 Hiroyuki Ohnaka

#ccc_r11

「テストがないコードはレガシーコードだ！」



#ccc_r11

「いまさら聞けない」

無料ダウンロードして、@ITの人気連載をまとめ読み！
第二弾は
「ゼロからのリレーショナルデータベース入門」
@IT eBookシリーズ
詳しくはこちら

» 2014年10月17日 18時00分 更新

いまさら聞けないTDD/BDD超入門（4）：

開発現場で保守性の高いTDD／BDDを実現するための3つのポイント――テストレベル／網羅性とは (1/3)

開発現場でTDD／BDDを導入するためのポイントを大きく三つに分けて解説。テストレベルや網羅性、サイクルタイムについても紹介します。

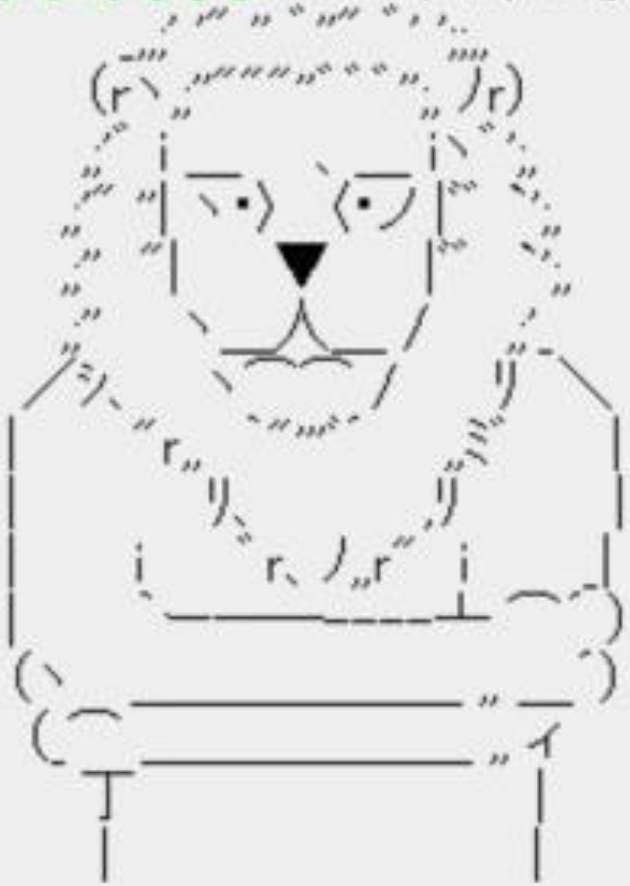
［きょん，STAR（テスト自動化研究会）］

印刷／PDF | ツイート 90 | いいね！ 79 | B! 103 | g+1 7 | Pocket 176 | メールで送信
類似記事の掲載をメールで通知 | 連載「いまさら聞けないTDD/BDD超入門」の新着をメールで通知
Google™カスタム検索
メルマガに登録する
Test & Tools
スポンサーからのお知らせ
データ活用「3つの要諦」を学ぶ
データサイエンティスト第一人者が語る、データをビジネス成果につげる3つのポイントとは
小室淑恵流ワークライフ・バランスの本質とは
納期に追われる技術者、客に振り回される営業そして@IT編集長…ワークワークを脱するには
万が一侵入されても被害は最小限に！
注目の「拡散防止型セキュリティモデル」とは？
PR 失敗から学ぶ、VMware環境のバックアップ設計ポイント
Webシステム化など、短距離実現法を伝授

http://www.atmarkit.co.jp/ait/articles/1410/17/news034.html

各所で猛威を振るうt_wada.png

名無し募集中。。。 : 2011/12/06(火) 18:32:24.37 0



テスト書いてないとかお前それ
@t_wadaの前でも
同じこと言えんの?

テストを書かなければプログラマーにあらず？

その一方で…

Copyright 2014 Hiroyuki Ohnaka

TDD is dead.

TDD is dead. Long live testing.

By David Heinemeier Hansson on April 23, 2014

Test-first fundamentalism is like abstinence-only sex ed: An unrealistic, ineffective morality campaign for self-loathing and shaming.

It didn't start out like that. When I first discovered TDD, it was like a courteous invitation to a better world of writing software. A mind hack to get you going with the practice of testing where no testing had happened before. It opened my eyes to the tranquility of a well-tested code base, and the bliss of confidence it grants those making changes to software.

The test-first part was a wonderful set of training wheels that taught me how to think about testing at a deeper level, but also some I quickly left behind.

http://david.heinemeierhansson.com/2014/tdd-is-dead-long-live-testing.html

#ccc_r11

「TDDを諦める」

高柴ラボ ▾ 読者になる　　記事を書く　setoazusa ▾

最新記事

- TDDを諦めることと、RSpecをやめること
- Capistrano2系で実行時にSSHのパスワード入力プロンプトが出ない問題の解決法あれこれ
- 誰でも簡単にオリジナルの見たまま編集エディタを作成できるライブラリ「wysiwyger」を作りました

プロフィール

id:takashibagura

読者になる 3

高柴ラボ

主にソフトウェア開発にまつわるポエムです

TDDを諦めることと、RSpecをやめること

開発手法　Ruby on Rails　RSpec　Ruby　2014-10-17

最近Web上でも仕事場でも、RSpecをやめて別のテストフレームワークに変えようと思っている……みたいな話をちょくちょく見聞きするようになった。僕がRuby on Railsで開発を始めた2012年8月当時、すでにRSpecはテストフレームワークのデファクトと言ってよかった。一斉を風靡したRSpecが、なぜ今見直され始めているのか。

きっかけになったのは今年4月の、Rails作者であるDavid Heinemeier Hansson(以下DHH)によるTDD is dead発言だと思う。

5月にはこの発言によるTDDへの風評被害を重く見たKent Beck*1が、レフリーにMartin Fowler*2を迎え、DHHと相対するドリームマッチが開催された。この会談の内容は、POSTDで全文日本語翻訳されている。

本エントリは翻訳リクエストより投稿いただきました。ありがとう

http://takashiba-labo.hatenablog.com/entry/2014/10/17/003431

「テスト駆動開発はなぜ流行らなかったのか？」

2ちゃんねる ★スマホ版★ ■掲示板に戻る■ 全部 1- 最新50

■ このスレッドは過去ログ倉庫に格納されています

テスト駆動開発はなぜ流行らなかったのか？2

1 :デフォルトの名無しさん:2014/01/13(月) 08:09:14.87
まとめ

適切にテストコードを導入するためには
適切なコードの書き方や設計まで知っておく必要があり
既存のコードが適切で無い場合、大幅な書き換えが必要になるから。
最悪開発プロセスから変えないといけない。

またテストコードが書きづらい部分(GUIやウェブUI等)が存在し
書きづらいということを知っていないと、無理に書こうとして
挫折するか、無理矢理なテストコードを書いてメンテナス不能になるから

2 :デフォルトの名無しさん:2014/01/13(月) 11:09:56.95
関数型言語で書くとテストコードが書きやすいコードに自然になるけど、
バカなドカタどもには使いこなせないから
テストコードが書き難いなんて低レベルな争いになる

3 :デフォルトの名無しさん:2014/01/13(月) 12:02:39.90
GUIやWeb は、2014年現在では、自動テストの導入コストが高過ぎる
ので手動でテストすることになる。
でも、詳細設計やコーディングを始める前に、テスト仕様(=検収条件)
を定義することはできるので、TDD の考え方は適用できる。

この場合、Control・Model層 の自動テストをきちんとやることはもちろん
View層が疎かつ薄くなるように設計し
人手による(高価な)GUI テストの範囲を限定する必要がある。

http://peace.2ch.net/test/read.cgi/tech/1389568154/

どうしてこうなった

- もともとTDD(というよりアジャイル開発)に関連するプラクティスはそれぞれが密結合していて、TDD単独で理解するのが難しいというのはありますが…

Copyright 2014 Hiroyuki Ohnaka

「テスト」と責任の関係

- テストという工程は、その性格上、品質保証や、成果物への責任と強い関係をもっている
- 「テストを書くべきだ」という論に明快に異を唱えられる人は少ない

Copyright 2014 Hiroyuki Ohnaka

- 開発プロセスの望ましい姿を論じていたはずが、道義的な責任の問題になってしまう
- テストを書く人と書かない人の間の感情的な対立は、何も生み出さない

Copyright 2014 Hiroyuki Ohnaka

「TDDやれば偉いわけじゃない」

t-wada　TDD, rspec

RSpec の話と TDD の話が混ざっている印象。個別のツールや狭義の TDD (TDD 三原則とか) に拘ったり怯えたりするのは生産的でないと思う。TDD やれば偉いわけじゃないし、やらないと悪いわけでもない。

2014/10/20 10:51　埋め込みコード　0　0　0　13

http://b.hatena.ne.jp/entry/232721484/comment/t-wada

Copyright 2014 Hiroyuki Ohnaka

幻想を捨てる

- カバレッジ100%
- ユニットテストを書けばバグがなくなる
- End to End のテストで全業務シナリオを網羅する

Copyright 2014 Hiroyuki Ohnaka

大事なことは、現場の開発プロセスの中で、TDDをどう位置づけるのか、開発プロセスの中でどうやって総合力を発揮するか

そのことを踏まえずに、TDDに意味があるかを考えても意味が無い

Copyright 2014 Hiroyuki Ohnaka

では、なぜTDDするのか

…?

Copyright 2014 Hiroyuki Ohnaka

一つの答え

- 「動作するきれいなコード」のために

Copyright 2014 Hiroyuki Ohnaka

テストファーストにこだわらない

- 表層的に、テストを先に書くかどうかではなく、テストを書くこととプロダクションコードを書くことが統合されているか
- そして、テストが開発の助けになっているか

Copyright 2014 Hiroyuki Ohnaka

100点を目指さない

- 隣の芝生は青く見える
- システム開発は勝ち負けではない

Copyright 2014 Hiroyuki Ohnaka

人を知る

- 「人間はほとんどの場合、まず感情で決定し、証拠を用いて決定を正当化する」
- 「人は変化に抵抗するのではなく、変化させられることに抵抗する」

「Fearless Change アジャイルに効く アイデアを組織に広めるための48のパターン」

Copyright 2014 Hiroyuki Ohnaka

アジェンダ

- なぜTDDするのか
- **どうTDDするか**
- 技術的な各論
- なぜTDDするのか(again)

Copyright 2014 Hiroyuki Ohnaka

プロジェクト概要

- 大手通信事業者の法人向けサービス開発
- メンバー　開始時11名→現在21名
- 開発プロセスはスクラム
- サービスインまでの開発期間7ヶ月

Copyright 2014 Hiroyuki Ohnaka

プロジェクト概要

- Backbone.js + Spring MVC + Spring Batch + Hibernate(JPA)
- ユニットテスト総ケース数3800ケース

Copyright 2014 Hiroyuki Ohnaka

立ち上げ時の状況

- クライアントの開発部隊＋インテグレーター2社で1チーム
- 初めて顔をあわせるメンバーが多い
- とにかく手探り

Copyright 2014 Hiroyuki Ohnaka

ペアプロ

- ペアを組んでユニットテストのコーチング
- とことんつきあう

Copyright 2014 Hiroyuki Ohnaka

テストファーストにこだわらない

- イテレーションの中で実装とテストが完結していれば、テストファーストでなくてもよいことに
- テストがあることのメリットを実感してもらう

Copyright 2014 Hiroyuki Ohnaka

ユニットテストを過信しない

- フェイルファストしてないユニットテストが、信頼性を欠くのは事実
- マニュアルでのリグレッションテストも重視
- ユニットテストから漏れるバグがあることを受け入れる

Copyright 2014 Hiroyuki Ohnaka

全部をテストしない

「サーバーサイドはばっちりテストを書きます。フロントエンドも自動テストして、end to end のテストもSeleniumで自動化します」

「もちつけ」

Copyright 2014 Hiroyuki Ohnaka

- まずはサーバサードのユニットテストの整備から
- サーバーサイドはAPIのエントリポイントとなるController/Serviceクラスについてはテストを網羅
- 後は必要に応じて

Copyright 2014 Hiroyuki Ohnaka

100点を目指さない。優先順位をつける

- 再現性、繰り返し可能(Repeatable)
- 独立していること(Independent)

Copyright 2014 Hiroyuki Ohnaka

スキル・ノウハウの共有

- スプリントごとにペアプロのペアを組み替え
- アトラシアンフルスタック(JIRA/Confluence/Bamboo/Stash)による情報共有
- 「伝言ゲームソリューション」

Copyright 2014 Hiroyuki Ohnaka

しかし問題も…

開発が進むにつれ、たまっていく技術的負債

Copyright 2014 Hiroyuki Ohnaka

技術的負債

- 意図がはっきりしないテスト
- テストメソッド名とやっていることが一致しないテスト
- コピペ
- モックの使いすぎ
- etc…

Copyright 2014 Hiroyuki Ohnaka

技術的負債の返済

- 改善サイクルをまわす
- スプリントごとの振り返り(KPT)
- Tryをちゃんとタスク化する、工数を割り振る

Copyright 2014 Hiroyuki Ohnaka

結果

- リリース判定会議をスムーズにクリア
- リリース直前に定時帰り
- サービスイン後も定期的にリリースを実施、クリティカルな不具合0件！

Copyright 2014 Hiroyuki Ohnaka

どうしてうまくいった？

- RESTfulのAPIを定義した上で、フロントエンドとサーバサイドでそれぞれ実装して、それを結合するというプロセスだったので、ユニットテストでサーバーサイドの動作を確認するというアプローチが自然だった
- Controller層とService層もModuleを分割していたので、同じ事が言える

Copyright 2014 Hiroyuki Ohnaka

「信頼で結ばれた共同体」(※)

- 「2人目」の存在
- 問題を指摘するとちゃんと次のスプリントにのせてくれる
- 人対人でなく、問題対私たち
- 「いいもの」をつくるという合意

※「組織パターン」から

Copyright 2014 Hiroyuki Ohnaka

アジェンダ

- なぜTDDするのか
- どうTDDするか
- **技術的な各論**
- なぜTDDするのか(again)

Copyright 2014 Hiroyuki Ohnaka

お品書き

- スローテスト
- データベースを使うテスト
- Controller層のテスト
- モック
- フロントエンド
- End to Endのテスト
- 作業をテストする

Copyright 2014 Hiroyuki Ohnaka

スローテスト怖い

スローテストによる開発リズムの悪化

→ビルドの通らないコードのpush

→CIの信頼性低下

→デリバリーのフロー崩壊

Copyright 2014 Hiroyuki Ohnaka

＿人人人人人人＿
＞　突然の死　＜
￣Y^Y^Y^Y^Y

よくあるスローテスト対策

- 組み込みデータベースを使う
- データベースをモックする

Copyright 2014 Hiroyuki Ohnaka

だがちょっとまってほしい

- そもそも、「データベースを使うと遅い」のであれば、データベースって業務に使えないです…
- 「データベースを使うテストは遅い」は都市伝説？

Copyright 2014 Hiroyuki Ohnaka

実例(その1)

Build summary | Tests | Commits | Artifacts | Lo

Test results

654 tests in total　　**39 seconds** taken in total.

See detailed test results for Default Job

Copyright 2014 Hiroyuki Ohnaka

実例(その2)

せとあずさ♂(眠い)
@setoazusa

テストケース総数530件(うちDBアクセスが180件くらい)でコンパイル・テスト・インスペクション・カバレッジ測定もやってビルドが1分以内で終わるって健闘してると思うの。

RETWEET 1
FAVORITES 2

8:16 PM - 24 Apr 2012

https://twitter.com/setoazusa/status/194746629159526400

- Java開発の場合は、スローテストの原因になるのは、コンテナの立ち上げ
- ユニットテストのあるべき姿(独立性)からは外れますが、Spring FrameworkのSpringJUnit4ClassRunnnerはContainerのContextをキャッシュする実装になっているので、初期化コストはおさえられる。

Copyright 2014 Hiroyuki Ohnaka

何がデータベースを使うテストを難しくするのか

- ネックとなるのはテストの実行時間よりも、テストデータのメンテナンスに必要なコストが失わせる、テストのリズム感

Copyright 2014 Hiroyuki Ohnaka

Excelを捨てよ、街へ出よう

- IDEで作業しているときに一々Excelたちあげるのたるいし…
- Excelでファイルを開いているためにIDEがビルドエラーになるのってあるあるですよね？
- そもそも分散バージョン管理しているとマージできない！

Copyright 2014 Hiroyuki Ohnaka

XMLだけど自動生成すれば問題ないよねっ

- 本当はYAMLなりJSONにしたかったのですが、DBUnitとの相性との関係で。
- テストデータ作成時は開発者のテストデータに一度投入した後、mavenプラグインでエクスポートする方法を採用
  - ただ、mavenでエクスポートするのはあまり使われてないみたい..

Copyright 2014 Hiroyuki Ohnaka

データベースのアサーション、どうやろう…

- データベースの更新系処理は、データベースのテーブル全体をアサーションするのがセオリー。
- しかし、テストに関係してない、更新されないデータまで期待値に含む必要があるため、テストデータのメンテナンスコストが増大する問題がある。

Copyright 2014 Hiroyuki Ohnaka

- データベースの更新・削除の前後で、テーブルの状態の差分を取り、その差分をアサーションするようにDBUnitをカスタマイズ。(デルタアサーション)
- このことのより、期待値のデータのメンテナンスコストを削減

Copyright 2014 Hiroyuki Ohnaka

データベースマイグレーションはTDDに必須

- 開発者の相互の環境がテスト実行に影響を与えないようにするため、開発者ごとにVagrantでデータベースを用意。
- ローカル/CI/開発環境/ステージング/本番のスキーマ構成を管理するため、MyBatis Migrationを使用。

Copyright 2014 Hiroyuki Ohnaka

MyBatis Migrationによるデータベース構成管理

```
C:¥Users¥azusa¥git¥migration¥migration¥scripts>more 20140204142208_dept.sql
--// dept
-- Migration SQL that makes the change goes here.


CREATE TABLE dept
(
   deptno numeric(2,0) NOT NULL,
   dname character varying(14),
   loc character varying(13),
   versionno numeric(8,0),
   active numeric(1,0),
   CONSTRAINT dept_pkey PRIMARY KEY (deptno)
)
WITH (
   OIDS=FALSE
);

--//@UNDO
-- SQL to undo the change goes here.

DROP TABLE DEPT;
```

Copyright 2014 Hiroyuki Ohnaka

Controller層のテスト

- End to Endになるべく近い条件でテストしたい
- Spring MVC Test FrameworkはWeb層のリクエストを高い精度でシミュレートできるため、これでControllerをテスト

Copyright 2014 Hiroyuki Ohnaka

Spring によるRESTfulAPIのテスト

```
mockMvc.perform(get("/company/fieldnotes.jp/users/hiroyuki@fieldnotes.jp"))
    .andExpect(jsonPath("$.id",is("hiroyuki@fieldnotes.jp")))
    .andExpect(jsonPath("$.firstName",is("Hiroyuki")))
    .andExpect(jsonPath("$.lastName",is("Ohnaka")))
    .andExpect(status().isOk());
```

Copyright 2014 Hiroyuki Ohnaka

モックの使い方

- モックライブラリはJMockitを使用
- どうモックを使うか
  - 極力　end to end で行う。
  - 層の切れ目のところで、積極的にモックを使う

Copyright 2014 Hiroyuki Ohnaka

両方に長所と短所がある

- モックを使わない…end to end に近い
- モックを使う…モックを使う箇所でAPIとして層を定義して分離できる

Copyright 2014 Hiroyuki Ohnaka

ただし、モックのしすぎに注意

- JMockitの場合、ローカル変数やstaticメソッドもモックできるため、モックが過剰になりがち
- SUTのロジックの大半をモックしていたケースも…

Copyright 2014 Hiroyuki Ohnaka

モックした場合のコンテナー汚染問題

- Dependency Injectionしたコンポーネント内のオブジェクトをモックした場合、Springのコンテナー内のオブジェクトが汚染される問題
- モックを使うテストケースに@DirtyContextをつければ回避できるが、テストが目に見えて遅くなるため @After のメソッドの中でオブジェクトを差し戻すように修正

Copyright 2014 Hiroyuki Ohnaka

フロントエンドのユニットテスト

- サーバーサイドのみのテストでもフロントエンドに良い効果は及ぼせる
- フロントエンドを改修する時に、サーバーサイドをテストでガードできているため、リグレッションの検証範囲を限定できる。

Copyright 2014 Hiroyuki Ohnaka

とはいうものの…

- 開発が進むにつれ、フロントエンドにテストがないことによるサーバーサイドとのリズムのずれが顕在化
  - 結合テストの時のテストの粒度のずれ
  - リファクタリングのリズム感のずれ
  - ブランチをマージする時のリスクのずれ

Copyright 2014 Hiroyuki Ohnaka

「安西先生、テストが書きたいです…」

Copyright 2014 Hiroyuki Ohnaka

フロントエンドのテストへの取り組み

- Mocha でModelのバリデーションのテストから、そこから徐々に拡大
- Casperjs+Mocha でRestfulのAPIをモックした状態での機能テスト

Copyright 2014 Hiroyuki Ohnaka

- もともとGruntでビルドするようにしていたため、テスティングフレームワークの組み込み自体は意外にすんなりいった
- ただ、テストの書き方は現在も試行錯誤中

Copyright 2014 Hiroyuki Ohnaka

end to end のシステムテスト

- リグレッションテストは、イテレーション(1週間ないし2週間)の最後に、開発メンバー全員で手動で行うことによって実施
- テストケースはTestLinkで管理

Copyright 2014 Hiroyuki Ohnaka

end to end　レベルのシステムテスト

- プロジェクト終盤から、Geb+Spockによる自動リグレッションテストに徐々に移行
- 現在は、自動リグレッションテストと手動リグレッションテストが共存している状態

Copyright 2014 Hiroyuki Ohnaka

デプロイ自動化の問題

- プロジェクトのミッションとしてDevOpsの推進があり、その一環として自動デプロイを実施
- 自動デプロイの実装時に、スクリプトの問題による不具合・リグレッションが多発

Copyright 2014 Hiroyuki Ohnaka

ServerSpecによるデプロイ作業の検証

ServerSpecでデプロイスクリプトの動作を検証し、CIサーバーから各環境に対してテストを実施することで事態を改善

Copyright 2014 Hiroyuki Ohnaka

```
describe service('jboss') do
    it { should be_enabled   }
    it { should be_running   }
  end
   describe command("wget -O - http://localhost:8080/service/status") do
      it { should return_exit_status 0 }
  end
  describe "setting.propetiesがデプロイされていること" do
      describe command("diff #{RESOURCES}/setting.properties
#{JBOSS_HOME}/properties/setting.properties") do
         it { should return_exit_status 0 }
      end
  end
```

Copyright 2014 Hiroyuki Ohnaka

- サーバー環境に対する操作等の正当性をテスティングフレームワークで担保するのは、リターンを得やすいためおすすめ
- 他にも、環境ごとのプロパティファイルの項目に不整合がないかの検証をCI環境で実施するなど(RSpecを使用)

Copyright 2014 Hiroyuki Ohnaka

アジェンダ

- なぜTDDするのか
- どうTDDするか
- 技術的な各論
- **なぜTDDするのか(again)**

Copyright 2014 Hiroyuki Ohnaka

荒みきったコード

疲弊しきった現場

爆弾処理のような
リリース

http://www.flickr.com/photos/okinawa-soba/2951808529/
http://www.flickr.com/photos/22719239@N04/2246462044/
http://ja.wikipedia.org/wiki/%E3%83%95%E3%82%A1%E3%82%A4%E3%83%AB:Bomb_neutralizing_EOD_9.jpg

もうウンザリです。何も改善できません。

Copyright 2014 Hiroyuki Ohnaka

もうウンザリです。何も改善できません。

「しかし、私たちにはプログラミングをする本当の理由があるはずです。」

「もし、自分や同僚がプログラミングを楽しめるなら、どんなシステムに取り組んでいるかは重要ではありません。そのシステムに対してきちんとした仕事ができるはずで、そうでなければ待っているのは落胆です。そうなってしまっては何の楽しみもありませんし、私たちはそんな目に遭うべきではありません。」

「レガシーコード改善ガイド」

Copyright 2014 Hiroyuki Ohnaka

なぜTDDするのか

私の場合…

Copyright 2014 Hiroyuki Ohnaka

「君にも君を動かしているものがあるでしょう？」

あなたの場合…

Copyright 2014 Hiroyuki Ohnaka

「君が質の高いソフトウェアを届けることは誰にも止められない。君が現場に立って、お客さんに向けてプロジェクトの状況と、プロジェクトに必要なことを誠実に伝えることも誰にも止められないんだ。」

「アジャイルサムライ」

Copyright 2014 Hiroyuki Ohnaka

「世界よ 変われ」(※)

- ミッションを共有する仲間が集うコミュニティで、
- プログラマが一人一人知恵を持ち寄り、議論し、勇気づける場が出来たときに、世界は変わる
- そう思う。

※いきものがかり 「NEW WORLD MUSIC」

Copyright 2014 Hiroyuki Ohnaka

あなたがコミュニティです

- キミのいる世界、わたしのいる世界
- 自分が完璧なエンジニアでないことを認める
- 問題を他人事にしない。真摯に向き合う

Copyright 2014 Hiroyuki Ohnaka

「信頼で結ばれた共同体」のために

- スキルの有無/キャリアの長短を人を計る尺度にしない
- IT技術者としてのバックグラウンドを尊重する
- ペアの意志決定を尊重する

Copyright 2014 Hiroyuki Ohnaka

最後に

- 正直言ってやってきたことは「TDDのようななにか」ですが、それでいいと思っています。
- 答えは、あなたの現場にあります。
- Have a nice Testing!
- ご静聴ありがとうございました。

Copyright 2014 Hiroyuki Ohnaka